

ASTRO 501 — HW3b Interferometry Data Analysis

In this HW, you will be carrying out a binary fit using archival optical interferometry data and comparing to the published results using the PMOIREd python library.

As for last assignment, you can either turn in your pdf of your completed notebook (on GradeScope) or just the figures/text requested as indicated by **P1** , **P2**, etc.

Due March 31, 2026.

ASTRO 501 — Binary fitting of HD 174881 with PMOIREd

Goal: Download the calibrated **L3 science-ready OIFITS** dataset for **HD 174881** from the **JMMC OIBD**, then use **PMOIREd** to perform a **binary fit** on the **UT 2007 Jul 07 MIRC** data. Finally, compare your fitted separation and position angle against the values reported in **Torres et al. (2024)**.

You can use chatgpt to help you, though it may not know this library especially well without guidance.

What you will do

1. Install `pmoired` .
2. Download the correct OIFITS dataset from OIBD.
3. Load the OIFITS file with `PMOIREd` .
4. Follow PMOIREd example 3 binary tutorial. Note you will need to somewhat modify the example to fit for the uniform-disk diameters of both the primary and secondary.
5. Compare with the literature values from Torres et al. (2024).

What you will turn in

- P1. Show uv coverage, V2 and closure phases
- P2. Show significance map of binary grid search
- P3. Show V2 and closure phase data along with best fit model results
- P4. Show model image of binary stars with comparison to Torres et al. (figure1)
- P5. Show comparison of your model fit parameters to Torres et al. (table 3)

External resources

- OI DB: <https://www-static-2019.jmmc.fr/oidb.htm>
- PMOIRE D repo: <https://github.com/amerand/PMOIRE D>
- PMOIRE D examples repo: https://github.com/amerand/PMOIRE D_examples
- Torres et al. (2024): <https://ui.adsabs.harvard.edu/abs/2024ApJ...977...43T/abstract>

0. Directory setup and manual download workflow

I recommend a directory structure like this:

```
HW3b/
  HW3b_starter.ipynb
  data/
    HD174881/
      <put the UT2007Jul07 L3 MIRC OIFITS file here>
```

OI DB search instructions

Go to the OI DB search page and do the following:

1. Search target: **HD 174881**
2. Instrument: **MIRC**
3. Data level: **L3 (science ready)**
4. Select the dataset for **UT 2007 Jul 07**
5. Download the OIFITS product
6. Place the downloaded `.fits` / `.oifits` file(s) in:

data/HD174881/

```
In [3]: # If needed, install PMOIRE D.
        # Uncomment ONE of the options below.

        %pip install pmoired
        #%pip install git+https://github.com/amerand/PMOIRE D

        # After installation, restart the kernel if Jupyter asks you to.
```

Requirement already satisfied: pmoired in /usr/local/anaconda/lib/python3.9/site-packages (1.2.11)

Requirement already satisfied: scipy>=1.10.0 in /usr/local/anaconda/lib/python3.9/site-packages (from pmoired) (1.12.0)

Requirement already satisfied: numpy>=1.24.0 in /usr/local/anaconda/lib/python3.9/site-packages (from pmoired) (1.26.4)

Requirement already satisfied: matplotlib>=3.8.0 in /usr/local/anaconda/lib/python3.9/site-packages (from pmoired) (3.8.0)

Requirement already satisfied: astropy>=6.0.0 in /usr/local/anaconda/lib/python3.9/site-packages (from pmoired) (6.0.1)

Requirement already satisfied: astroquery>=0.4.4 in /usr/local/anaconda/lib/python3.9/site-packages (from pmoired) (0.4.10)

Requirement already satisfied: pyerfa>=2.0.1.1 in /usr/local/anaconda/lib/python3.9/site-packages (from astropy>=6.0.0->pmoired) (2.0.1.5)

Requirement already satisfied: astropy-iers-data>=0.2024.2.26.0.28.55 in /usr/local/anaconda/lib/python3.9/site-packages (from astropy>=6.0.0->pmoired) (0.2026.3.16.0.53.33)

Requirement already satisfied: PyYAML>=3.13 in /usr/local/anaconda/lib/python3.9/site-packages (from astropy>=6.0.0->pmoired) (6.0.1)

Requirement already satisfied: packaging>=19.0 in /usr/local/anaconda/lib/python3.9/site-packages (from astropy>=6.0.0->pmoired) (23.1)

Requirement already satisfied: requests>=2.19 in /usr/local/anaconda/lib/python3.9/site-packages (from astroquery>=0.4.4->pmoired) (2.31.0)

Requirement already satisfied: beautifulsoup4>=4.8 in /usr/local/anaconda/lib/python3.9/site-packages (from astroquery>=0.4.4->pmoired) (4.12.2)

Requirement already satisfied: html5lib>=0.999 in /usr/local/anaconda/lib/python3.9/site-packages (from astroquery>=0.4.4->pmoired) (1.1)

Requirement already satisfied: keyring>=15.0 in /usr/local/anaconda/lib/python3.9/site-packages (from astroquery>=0.4.4->pmoired) (25.6.0)

Requirement already satisfied: pyvo>=1.5 in /usr/local/anaconda/lib/python3.9/site-packages (from astroquery>=0.4.4->pmoired) (1.6.2)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/anaconda/lib/python3.9/site-packages (from matplotlib>=3.8.0->pmoired) (1.2.0)

Requirement already satisfied: cycler>=0.10 in /usr/local/anaconda/lib/python3.9/site-packages (from matplotlib>=3.8.0->pmoired) (0.11.0)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/anaconda/lib/python3.9/site-packages (from matplotlib>=3.8.0->pmoired) (4.25.0)

Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/anaconda/lib/python3.9/site-packages (from matplotlib>=3.8.0->pmoired) (1.4.4)

Requirement already satisfied: pillow>=6.2.0 in /usr/local/anaconda/lib/python3.9/site-packages (from matplotlib>=3.8.0->pmoired) (10.2.0)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/anaconda/lib/python3.9/site-packages (from matplotlib>=3.8.0->pmoired) (3.0.9)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/anaconda/lib/python3.9/site-packages (from matplotlib>=3.8.0->pmoired) (2.8.2)

Requirement already satisfied: importlib-resources>=3.2.0 in /usr/local/anaconda/lib/python3.9/site-packages (from matplotlib>=3.8.0->pmoired) (6.1.1)

Requirement already satisfied: soupsieve>1.2 in /usr/local/anaconda/lib/python3.9/site-packages (from beautifulsoup4>=4.8->astroquery>=0.4.4->pmoired) (2.5)

Requirement already satisfied: six>=1.9 in /usr/local/anaconda/lib/python3.9/site-packages (from html5lib>=0.999->astroquery>=0.4.4->pmoired) (1.16.0)

Requirement already satisfied: webencodings in /usr/local/anaconda/lib/python3.9/site-packages (from html5lib>=0.999->astroquery>=0.4.4->pmoired) (0.5.1)

Requirement already satisfied: zipp>=3.1.0 in /usr/local/anaconda/lib/python3.9/site-packages (from importlib-resources>=3.2.0->matplotlib>=3.8.0->pmoired) (3.11.0)

Requirement already satisfied: importlib_metadata>=4.11.4 in /usr/local/anaconda/lib/python3.9/site-packages (from keyring>=15.0->astroquery>=0.4.4->pmoired) (6.0.0)

Requirement already satisfied: jaraco.classes in /usr/local/anaconda/lib/python3.9/site-packages (from keyring>=15.0->astroquery>=0.4.4->pmoired) (3.4.0)

Requirement already satisfied: jaraco.funcitools in /usr/local/anaconda/lib/python3.9/site-packages (from keyring>=15.0->astroquery>=0.4.4->pmoired) (4.1.0)

Requirement already satisfied: jaraco.context in /usr/local/anaconda/lib/python3.9/site-packages (from keyring>=15.0->astroquery>=0.4.4->pmoired) (6.0.1)
 Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/anaconda/lib/python3.9/site-packages (from requests>=2.19->astroquery>=0.4.4->pmoired) (2.0.4)
 Requirement already satisfied: idna<4,>=2.5 in /usr/local/anaconda/lib/python3.9/site-packages (from requests>=2.19->astroquery>=0.4.4->pmoired) (3.4)
 Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/anaconda/lib/python3.9/site-packages (from requests>=2.19->astroquery>=0.4.4->pmoired) (1.26.18)
 Requirement already satisfied: certifi>=2017.4.17 in /usr/local/anaconda/lib/python3.9/site-packages (from requests>=2.19->astroquery>=0.4.4->pmoired) (2025.8.3)
 Requirement already satisfied: more_itertools in /usr/local/anaconda/lib/python3.9/site-packages (from jaraco.classes->keyring>=15.0->astroquery>=0.4.4->pmoired) (8.12.0)
 Requirement already satisfied: backports.tarfile in /usr/local/anaconda/lib/python3.9/site-packages (from jaraco.context->keyring>=15.0->astroquery>=0.4.4->pmoired) (1.2.0)
 Note: you may need to restart the kernel to use updated packages.

Following PMOIRE Example 3: AX Cir and companion search

<https://htmlpreview.github.io/?>

https://github.com/amerand/PMOIRED_examples/blob/main/html/EX3%20companion%20search%20AXC

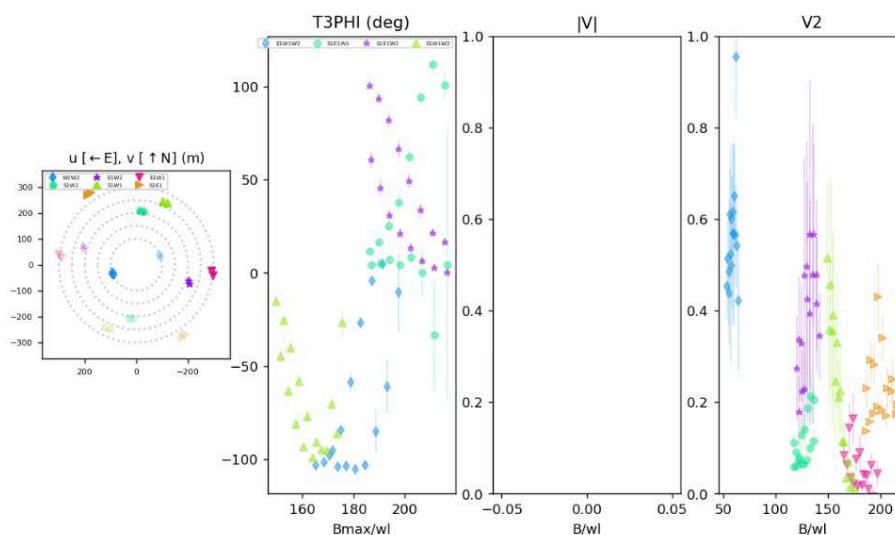
Only in our case, you will use the oifits data for HD174881 you downloaded in the earlier step.

In the cells below, I give an example of the plot or table you need to create for the assignment. Your plot doesn't have to look identical but should contain the same basic information.



P1. Show uv coverage, V2 and closure phases

Using the built-in PMOIRED plotting utilities, show the uv coverage, vis2, and closure phases in the HD174881 data file. This is just data not fits done yet. Here is an example:

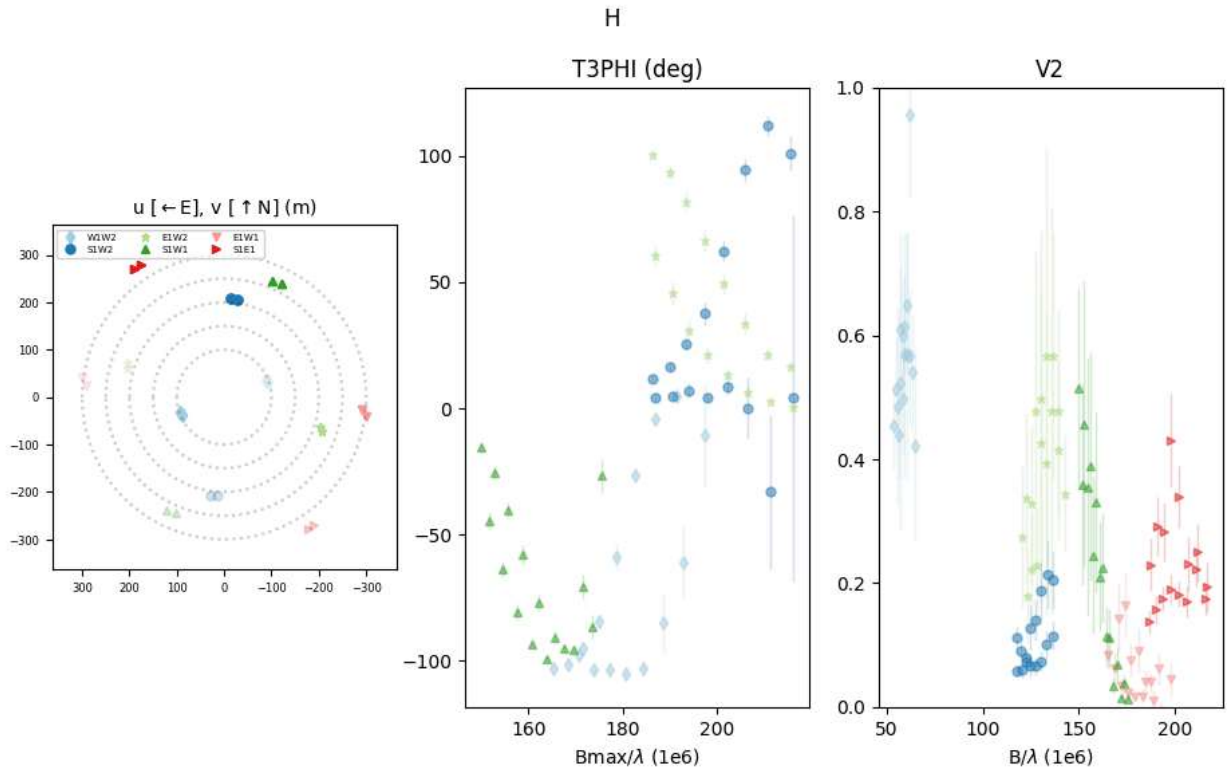


```
In [2]: import numpy as np
import os
import matplotlib.pyplot as plt
import astropy.io.fits as fits
```

```
import pmoired

oi = pmoired.OI("data/HD174881/MIRC.2007Jul07.HR_7112.v1.oifits")
print(oi)
oi.show(obs=['T3PHI', 'V2'])
```

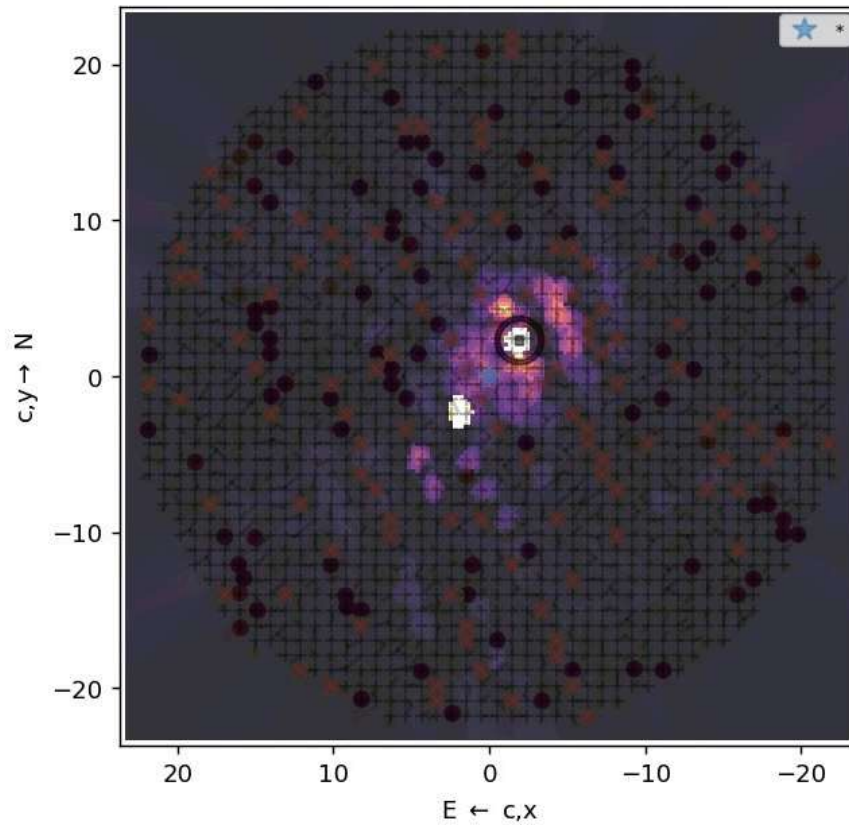
```
loadOI: loading data/HD174881/MIRC.2007Jul07.HR_7112.v1.oifits
> insname: "H_PRISM" targname: "HR_7112" pipeline: ""
> MJD: (2,) [54288.2518..54288.2688] ~2007-07-07T06:14:51.523
> E1-S1-W1-W2 | WL: (8,) [ 1.527 .. 1.768 ] um (R~48 P~48) {'T3': 4, 'VIS': 6, 'VIS2': 6}
| TELL: False
<pmoired.OI object at 0x000002AEF0A00E50>
```



P2. Show significance map of binary grid search

After following the fitting procedures in the example, include your gridsearch result showing binary detection significance map. Should show a clear detection of the companion similar (but not identical) to below:

Coarse grid-search significance map



```
In [28]: # -- smallest lambda/B in mas (first data set)
step = 180 * 3600 * 1000e-6 / np.pi / max([np.max(oi.data[0]['OI_VIS2'][k]['B/wl']) for k in range(oi.data[0]['WL'].size)])
R = np.mean(oi.data[0]['WL'] / oi.data[0]['dWL'])
print("step:", step)
print("R:", R)
print("grid size:", R)
lim_outer = 10
lim_inner = step/2

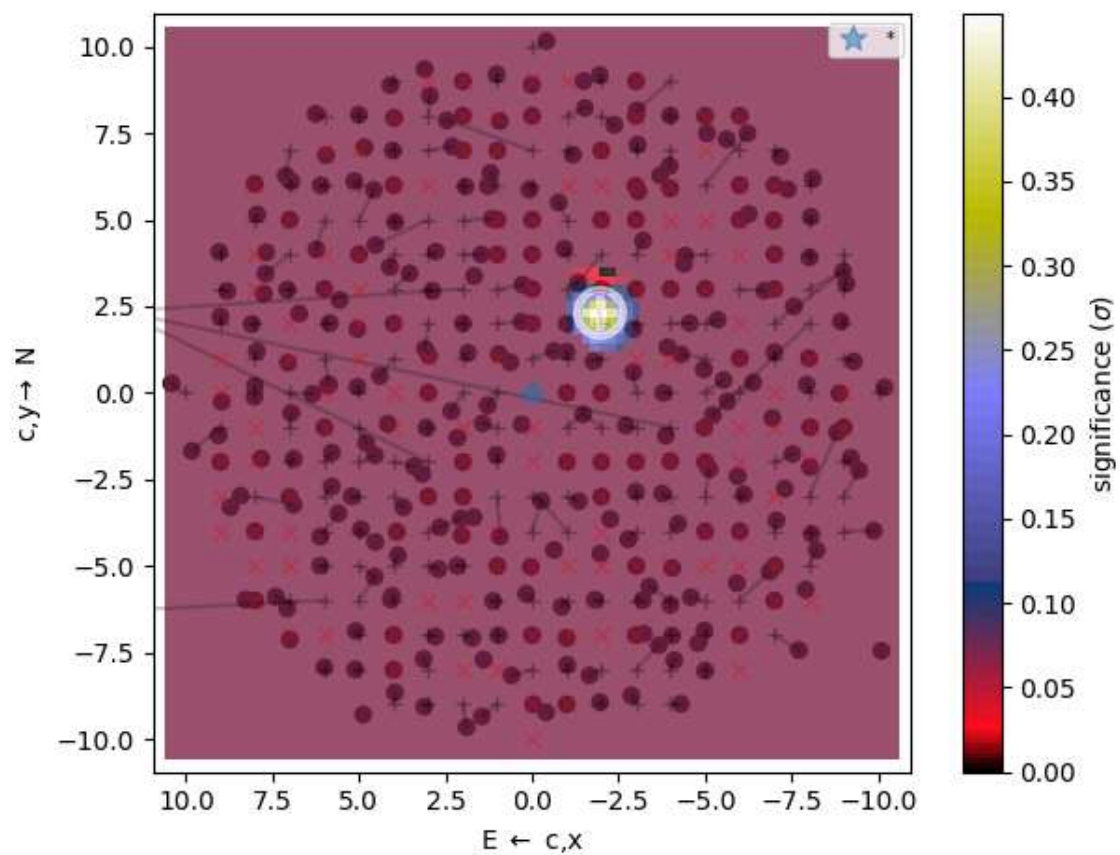
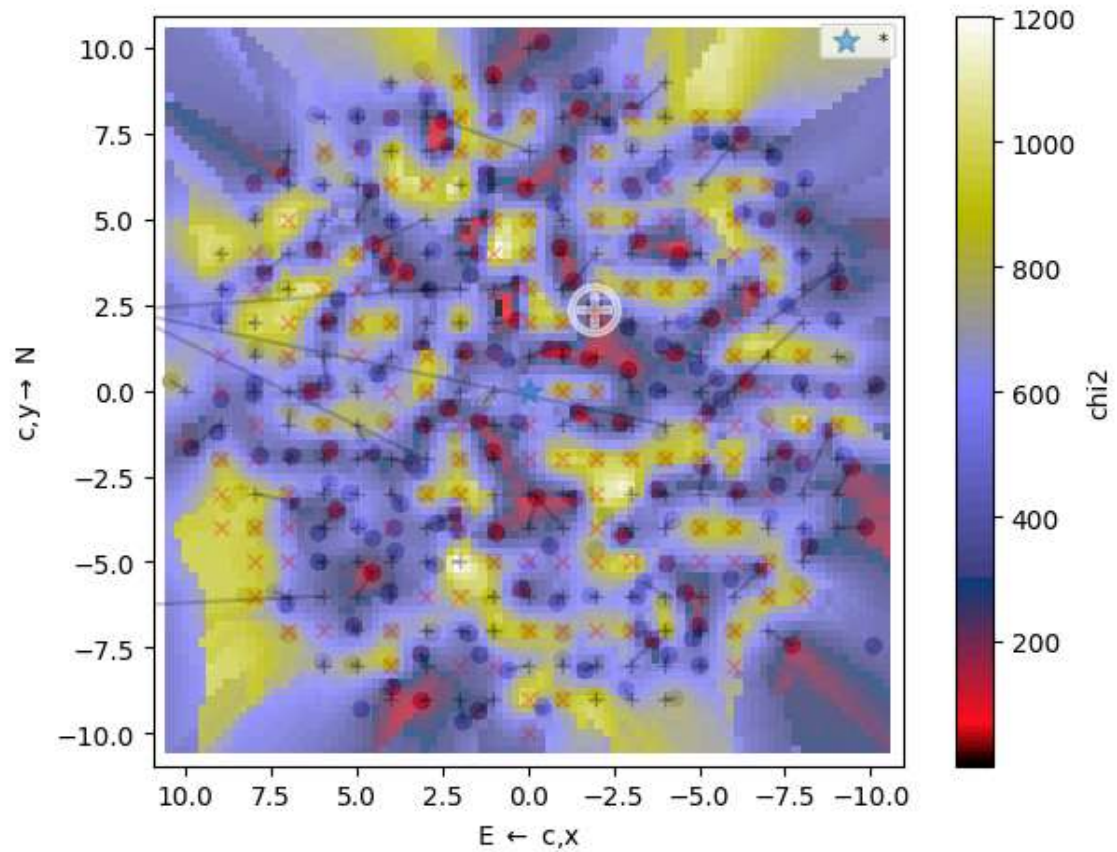
expl = {'grid':{'c,x':(-lim_outer, lim_outer, step), 'c,y':(-lim_outer, lim_outer, step)}}

# -- initial model dict: 'c,x' and 'c,y' do not matter, as they will be explored in the fit
param = {'*,ud':0.2, '*,f':1, 'c,ud':0.05, 'c,f':0., 'c,x':0, 'c,y':0}

oi.setupFit({'obs': ['T3PHI', 'V2']})

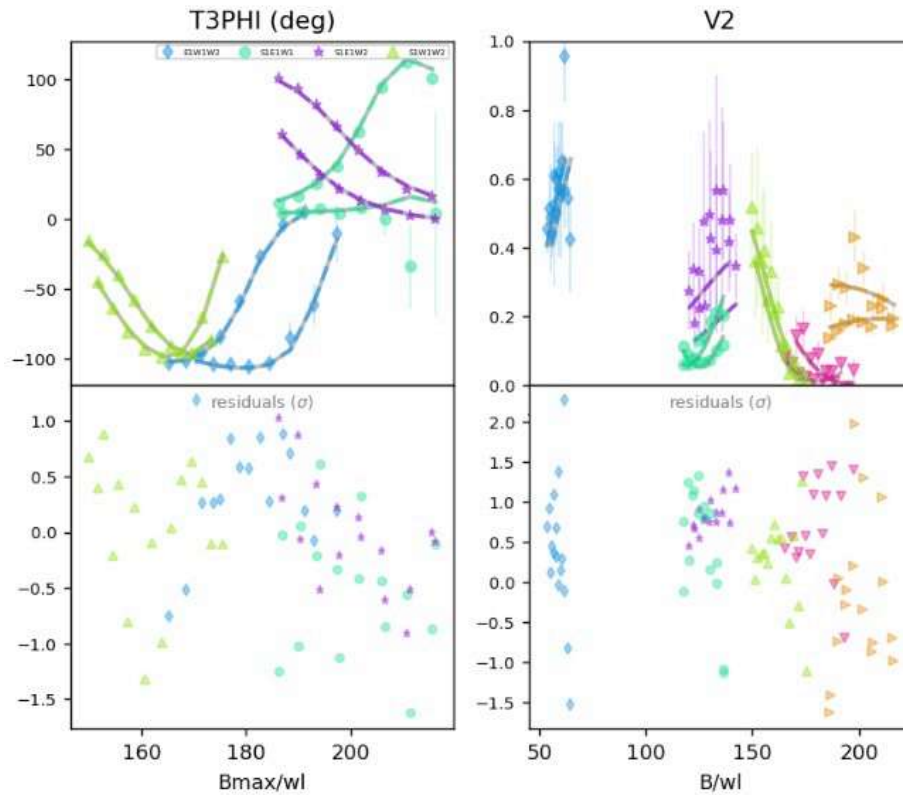
# -- actual grid fit
oi.gridFit(expl, model=param, doNotFit=['*,f'], constrain=[
    ('np.sqrt((c,x)**2 + (c,y)**2)', '<=', lim_outer),
    ('np.sqrt((c,x)**2 + (c,y)**2)', '>', lim_inner),
])

model = oi.bestfit['best'].copy()
# -- show the 2D grid of reduced chi2
oi.showGrid(interpolate=True, tight=True)
bestUD = oi.bestfit['best']
oi.showGrid(interpolate=True, tight=True, significance=bestUD)
```

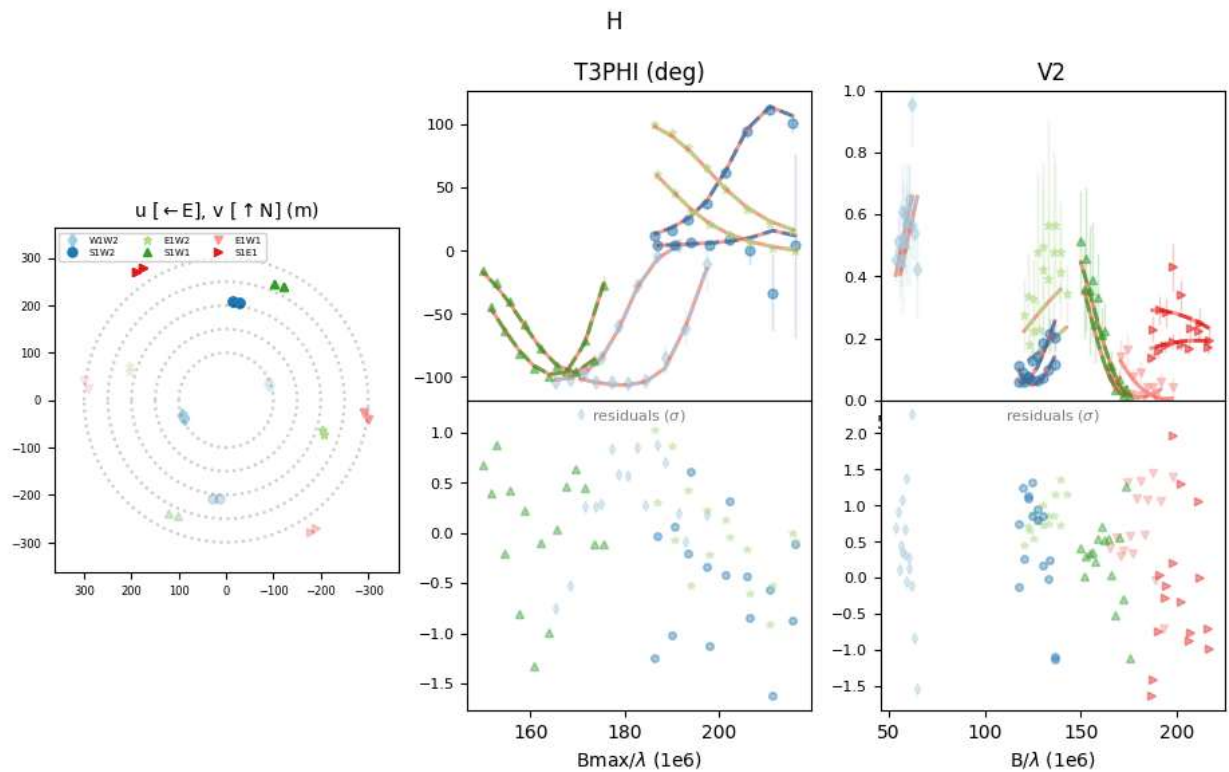



P3. Show V2 and closure phase data along with best fit model results

After finding the best fit, show the results compared to the data. Should look similar to this:

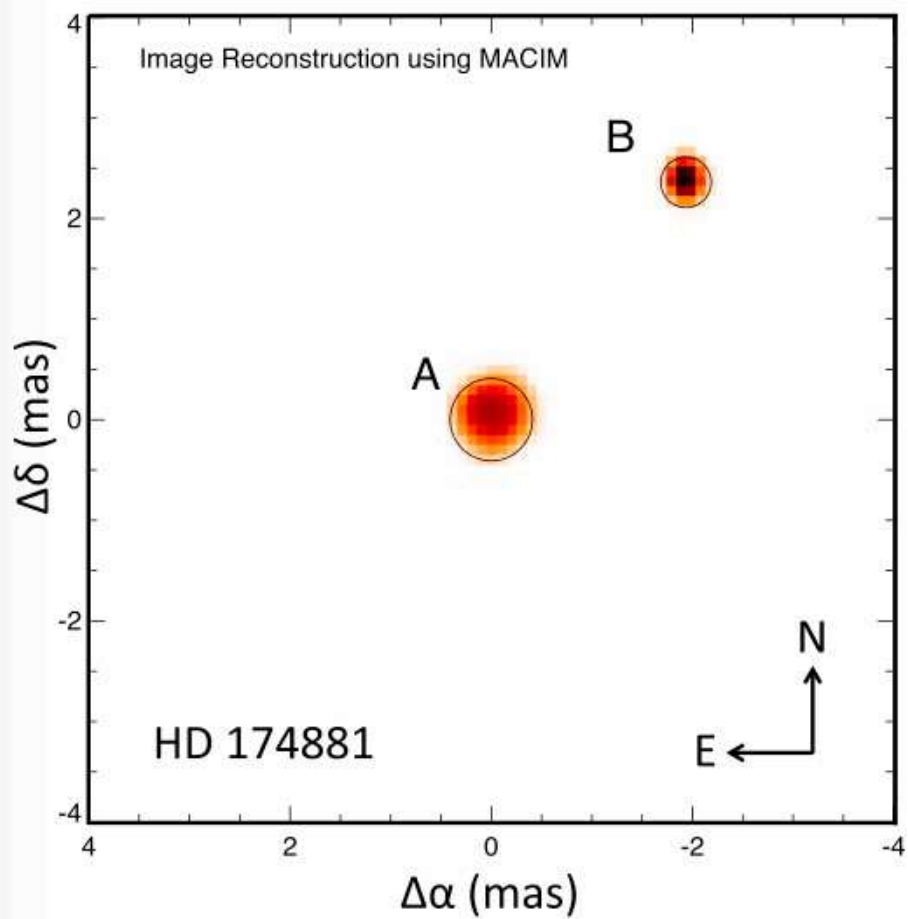
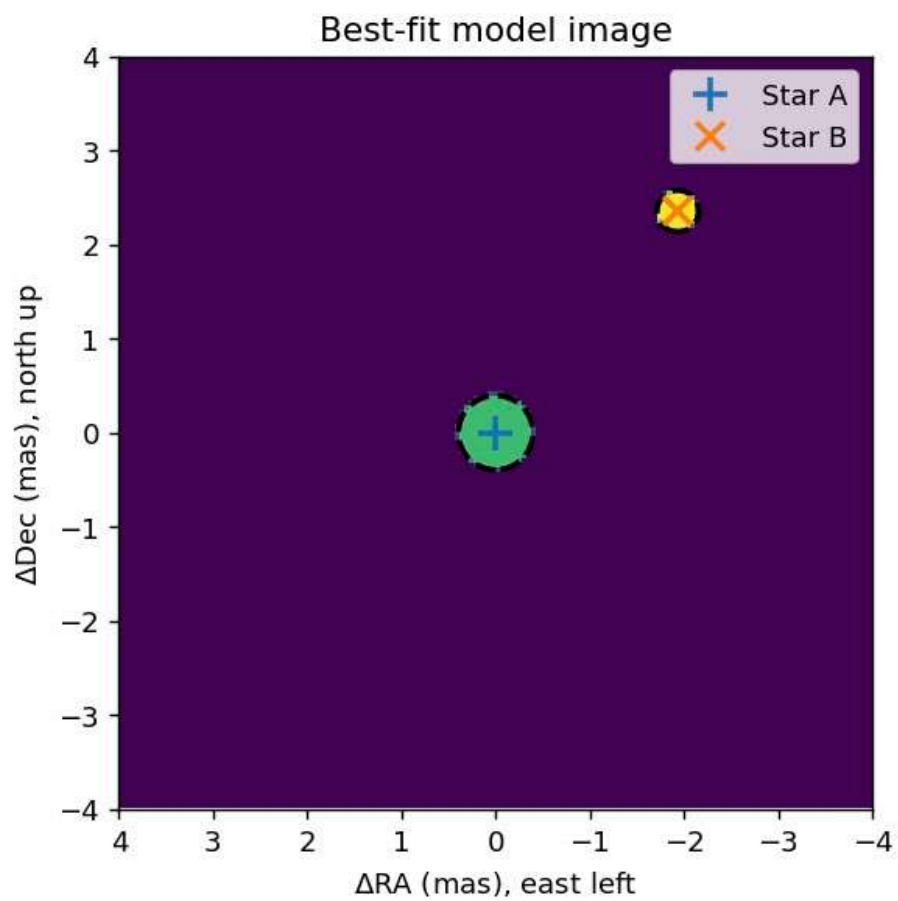


```
In [29]: # We already showed it, but here it is again:
oi.show(
    model='best',
    spectro=False,
    showUV=True,
    perSetup=True,
    imFov=None,
    imMax=0.05,
)
```



P4. Show model image of binary stars with comparison to Torres et al. (figure1)

Compare the model image (two uniform disks) to the paper model image. For this part, you will need to deviate from the example and fit for both the primary and secondary uniform disk sizes. Should look similar to below:



```

In [34]: # P4: two-UD image built from fitted parameters
params = model

a_ud = params.get("*,ud",np.nan)
b_ud = params.get("c,ud",np.nan)
b_x = params.get("c,x",np.nan)
b_y = params.get("c,y",np.nan)
a_f = params.get("*,y",1.0) # We previously fixed it
b_f = params.get("c,f",0.3)

npix = 1024
axis = np.linspace(-4, 4, npix)
X, Y = np.meshgrid(axis, axis)

img = np.zeros_like(X, dtype=float)
img[(X**2 + Y**2) <= (0.5 * a_ud) ** 2] += a_f
img[((X - b_x) ** 2 + (Y - b_y) ** 2) <= (0.5 * b_ud) ** 2] += b_f
if img.max() > 0:
    img /= img.max()

fig, ax = plt.subplots(1, 2, figsize=(12,6), constrained_layout=False)
im = ax[0].imshow(img, origin='lower',extent=[-4, 4, -4, 4],cmap='hot_r')
ax[0].set_aspect('equal')
ax[0].set_title('PMOIRE model')
ax[0].set_xlabel(r'$\Delta \alpha$ (mas)')
ax[0].set_ylabel(r'$\Delta \delta$ (mas)')
ax[0].invert_xaxis()

xs = [0.0]
ys = [0.0]

if np.isfinite(b_x) and np.isfinite(b_y):
    xs.append(b_x)
    ys.append(b_y)

ax[0].scatter(xs, ys, c=['cyan', 'lime'][:len(xs)], marker='x', s=45)

for xc, yc, d, color, label in [
    (0.0, 0.0, a_ud, 'cyan', 'A,ud'),
    (b_x, b_y, b_ud, 'lime', 'B,ud'),
]:
    if np.isfinite(xc) and np.isfinite(yc) and np.isfinite(d) and d > 0:
        ax[0].add_patch(
            plt.Circle((xc, yc), 0.5 * d, fill=False, ec=color, lw=1.5, label=label)
        )

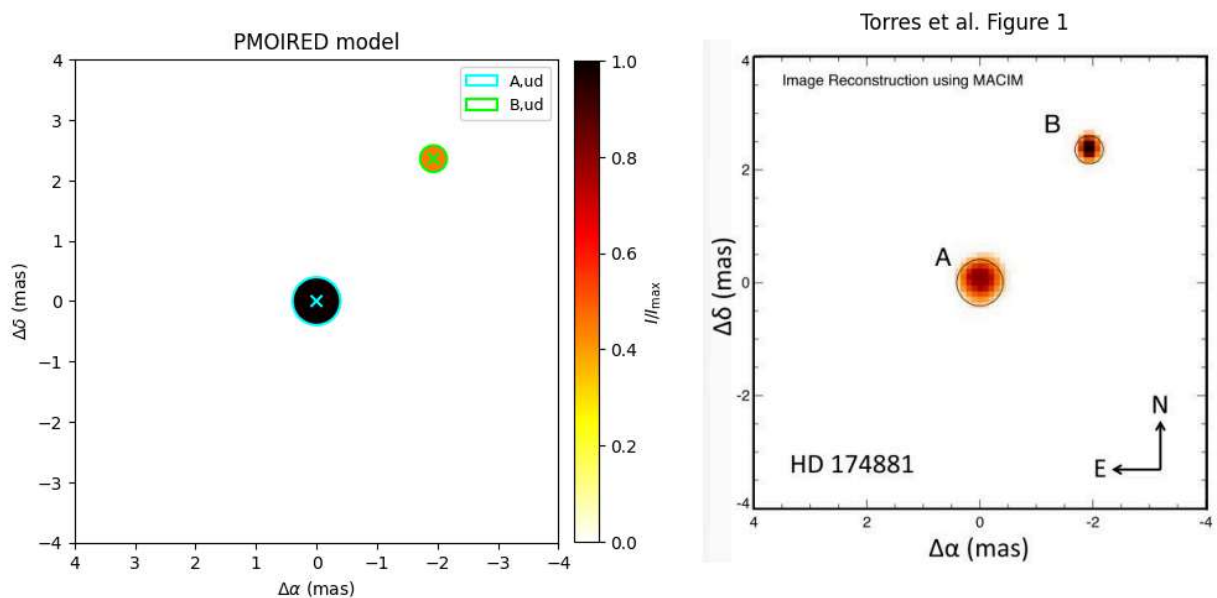
ax[0].legend(loc='upper right', fontsize=9)

cb = plt.colorbar(im, ax=ax[0], fraction=0.046, pad=0.03)
cb.set_label('$I/I_{\mathrm{max}}$')

ref = plt.imread("figs/binary_paper.jpg")
ax[1].imshow(ref)
ax[1].axis('off')
ax[1].set_title('Torres et al. Figure 1')

```

Out[34]: Text(0.5, 1.0, 'Torres et al. Figure 1')



P5. Show comparison of your model fit parameters to Torres et al. (table 3)

Lastly, compare your results (with error bars) to the published parameters. There is a oifits bootstrap method that provides error analysis. For full credit, you should find a way to include the primary and secondary uniform disk sizes along with the flux ratio and separation. For instance, here are some useful parameter keywords.

```
"*,ud": 1.0, # starting guess for primary uniform-disk diameter [mas]
"*,f": 1.0, # fixed reference flux [Always keep fixed as its a
reference]
"c,f": 0.3, # initial flux ratio scale guess
"c,x": 0.0, # starting east offset [mas]
"c,y": 0.0, # starting north offset [mas]
"c,ud": 0.0, # companion uniform disk diameter [mas]
```

"* = star" "c = companion"

Here is my comparison table as an example. You can use (delta RA, delta DEC) instead of (separation, PA) if you find it easier.

Comparison with Torres et al. (2024)

```
sep: fit = 3.0497 mas, paper = 3.0510 mas, delta = -0.0013 mas
PA: fit = 320.64 deg, paper = 320.64 deg, delta = -0.00 deg
A diam: fit = 0.7920 mas, paper = 0.8170 ± 0.0300 mas, delta = -0.0250 mas (-0.83σ)
B diam: fit = 0.4416 mas, paper = 0.5000 ± 0.0500 mas, delta = -0.0584 mas (-1.17σ)
B/A flux: fit = 0.4532, paper = 0.4580 ± 0.0030, delta = -0.0048 (-1.61σ)
Bootstrap medians for comparison:
sep: 3.0495 -0.0021 +0.0016 (paper 3.0510)
pa: 320.6409 -0.0274 +0.0271 (paper 320.6400)
*,ud: 0.7905 -0.0060 +0.0055 (paper 0.8170)
c,ud: 0.4445 -0.0153 +0.0164 (paper 0.5000)
c,f: 0.4545 -0.0043 +0.0058 (paper 0.4580)
```

```
In [39]: oi.bootstrapFit(1000, verbose=0)

sep_fit = np.sqrt(b_x**2+b_y**2)
pa_fit = np.degrees(np.arctan2(b_x, b_y)) % 360
```

```

print('Comparison with Torres et al. (2024)')
print('-----')
print(f'\tsep: fit = {sep_fit:.4f} mas, paper = 3.0510 mas, delta = {sep_fit-3.0510:.4f} mas')
print(f'\tPA: fit = {pa_fit:.4f} deg, paper = 320.64 deg, delta = {pa_fit-320.64:.2f} deg')
print(f'\tA diam: fit = {a_ud:.4f} mas, paper = 0.8170±0.0300 mas, delta = {a_ud-0.8170:.4f} mas')
print(f'\tB diam: fit = {b_ud:.4f} mas, paper = 0.5000±0.0500 mas, delta = {b_ud-0.5:.4f} mas')
print(f'\tB/A flux: fit = {b_f:.4f}, paper = 0.4580±0.0030, delta = {b_f-0.458:.4f} ({(b_f-0.458)/0.0030})')

boot = oi.boot
cx_s = np.array(oi.boot['all best']['c,x'])
cy_s = np.array(oi.boot['all best']['c,y'])
a_ud_s = np.array(oi.boot['all best']['*,ud'])
c_ud_s = np.array(oi.boot['all best']['c,ud'])
f_s = np.array(oi.boot['all best']['c,f'])
sep_s = np.sqrt(cx_s**2 + cy_s**2) # mas
pa_s = np.degrees(np.arctan2(cx_s, cy_s)) % 360

def get_err(samples):
    med = np.median(samples)
    lo = med - np.percentile(samples, 16)
    hi = np.percentile(samples, 84) - med
    return med, lo, hi

sep_med, sep_lo, sep_hi = get_err(sep_s)
pa_med, pa_lo, pa_hi = get_err(pa_s)
a_med, a_lo, a_hi = get_err(a_ud_s)
b_med, b_lo, b_hi = get_err(c_ud_s)
f_med, f_lo, f_hi = get_err(f_s)

print('Bootstrap medians for comparison:')

print(f'\tsep: {sep_med:.4f} -{sep_lo:.4f} +{sep_hi:.4f}\t(paper 3.0510)')
print(f'\tPA: {pa_med:.4f} -{pa_lo:.4f} +{pa_hi:.4f}\t(paper 320.6400)')
print(f'\t*,ud: {a_med:.4f} -{a_lo:.4f} +{a_hi:.4f}\t(paper 0.8170)')
print(f'\tc,ud: {b_med:.4f} -{b_lo:.4f} +{b_hi:.4f}\t(paper 0.5000)')
print(f'\tc,f: {f_med:.4f} -{f_lo:.4f} +{f_hi:.4f}\t(paper 0.4580)')

```

Comparison with Torres et al. (2024)

```

-----
sep: fit = 3.0497 mas, paper = 3.0510 mas, delta = -0.0013 mas
PA: fit = 320.6365 deg, paper = 320.64 deg, delta = -0.00 deg
A diam: fit = 0.7920 mas, paper = 0.8170±0.0300 mas, delta = -0.0250 mas (-0.83σ)
B diam: fit = 0.4417 mas, paper = 0.5000±0.0500 mas, delta = -0.0583 mas (-1.17σ)
B/A flux: fit = 0.4532, paper = 0.4580±0.0030, delta = -0.0048 (-1.59σ)
Bootstrap medians for comparison:
sep: 3.0495 -0.0021 +0.0017      (paper 3.0510)
PA: 320.6373 -0.0304 +0.0298     (paper 320.6400)
*,ud: 0.7910 -0.0108 +0.0046     (paper 0.8170)
c,ud: 0.4426 -0.0246 +0.0168     (paper 0.5000)
c,f: 0.4538 -0.0041 +0.0053     (paper 0.4580)

```

In []: